

Persistent

可持久化数据结构简介

简介

可持久化数据结构 (Persistent data structure) 总是可以保留每一个历史版本，并且支持操作的不可变特性 (immutable)。

可持久化分类

部分可持久化 (Partially Persistent)

所有版本都可以访问，但是只有最新版本可以修改。

完全可持久化 (Fully Persistent)

所有版本都既可以访问又可以修改。

若支持将两个历史版本合并，则又称为 Confluently Persistent

实际应用

几何计算

在几何计算中有许多离线算法，如扫描线算法一次扫过去回答所有询问，在时间复杂度分析上相当优异。但强迫在线的情况下，每一次都扫描一次，询问操作的时间复杂度就从对数时间降成线性。为了解决这一种情况，持久化技术给了另一种思维，我们将扫描线的时间轴作为一个变动依据，持久化相关的结构，只要我们能将询问在对数时间内穿梭于这个时间轴，必能动态解决先前的问题。

字串处理

为了达到非常高效率的合并操作，防止大量重复性字串的生成伴随的效能退化，使得各方面的操作都能远低于线性操作。如 C++ rope 就是一个持久化的数据结构。不只是字串操作，若处理类型有大量重复的情况，持久化的概念便能派上用场。

版本回溯

实际上就是对应大部分的应用软体中的 redo/undo。如果资料库/操作变动为了高效率操作而会配上复杂的结构（并不像 hash, set 反转操作只需要常数或对数时间），那么为了快速回推变动结果，持久化结构就是要减少 redo/undo 的花费。

资料库本身可以常数回推，纪录变动的部分情况即可。而应用层的计算，大部分实作都是砍掉快取，并且重新计算出一份新的结构，有时候回推的变动大小为 m ，为了重新计算结构而消耗了 $n+m$ ，如果 n 和 m 的差距非常大，那连续回推的体感就很糟糕。

函数式编程

函数式编程需要特别的数据结构以符合语言特性，其中不可变的性质更为重要，以利于并行环境与除错。如面向对象编程的 Java 8 后引入 stream 类，支援写出函数式的语法设计，可提供惰性求值、无限值域等的特殊功能。

参考

- https://en.wikipedia.org/wiki/Persistent_data_structure
 - MIT 课程 <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-854j-advanced-algorithms-fall-2005/lecture-notes/persistent.pdf>
-

本页面最近更新：2023/2/18 07:57:07, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [morris821028](#), [Enter-tainer](#), [GavinZhengOI](#), [lr1d](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用